CEBU INSTITUTE OF TECHNOLOGY
UNIVERSITY

IT342-Section
SYSTEMS INTEGRATION AND ARCHITECTURE 1

FUNCTIONAL REQUIREMENTS SPECIFICATION (FRS)

Project Title: BrainBox - A Centralized Learning and Knowledge Tool

Prepared By: Joana Carla Gako

Date of Submission: January 30, 2026

Version: 1

Table of Contents

1. Introduction

1.1. Purpose

This document describes the functional and non-functional requirements of BrainBox. It is intended for the project developer, instructor, testers, and evaluators who need to understand what the system must do and what constraints the system must satisfy.

1.2. Scope

BrainBox is a multi-platform study application that allows users to manage notebooks, generate and study quizzes and flashcards, organize study materials, use AI-assisted notebook features, maintain playlists and playback queues, and recover previous notebook content through version history.

The system includes a Spring Boot backend API, a React/Vite web application, and an Android/Kotlin mobile application. The system covers authentication, profile and settings management, notebook management, notebook content versioning, categories, AI configuration and conversations, quizzes, flashcards, playlists, playback queue, and mobile audio playback.

The system does not cover payment processing, real-time collaborative editing, public social sharing, production analytics dashboards, offline mobile study mode, or admin management workflows.

1.3. Definitions, Acronyms, and Abbreviations

Term	Definition
AI	Artificial intelligence support used for notebook assistance and generated study materials.
API	Application Programming Interface.
Client Mutation ID	Client-generated identifier used to make selected write operations idempotent.
DTO	Data Transfer Object used for API request and response payloads.
JWT	JSON Web Token used for authenticated API access.
SDD	System Design Document.

SRS	Software Requirements Specification.
Version History	Saved notebook content snapshots that users can list, preview, and restore.

2. Overall Description

2.1. System Perspective

BrainBox is a client-server system. The web and mobile clients communicate with the Spring Boot backend through REST API endpoints. The backend handles authentication, user-owned study data, generated study content, AI configuration, notebook versioning, and playback queue state. The web application provides the full notebook editor, profile/settings surfaces, study management screens, and playback queue UI. The Android application provides authenticated mobile screens, quiz and flashcard study screens, playlist screens, profile access, and audio playback behavior through backend-provided data.

2.2. User Classes and Characteristics

User Class	Characteristics
Guest User	Can register, verify email, log in, use Google sign-in, and request password recovery.
Authenticated User	Can manage profile, notebooks, notebook versions, categories, AI settings, AI conversations, quizzes, flashcards, playlists, playback queue, exports, and mobile study flows.
External Service	Email and AI provider services used by the backend for verification, reset, and AI workflows.

2.3. Operating Environment

Component	Operating Environment
Backend	Java 21, Spring Boot, Maven, PostgreSQL, REST over HTTP.
Web Frontend	React, Vite, React Router, TanStack Query, TipTap editor, modern browser.

Mobile Application	Android API 24 or higher, Kotlin, Jetpack Compose, Retrofit, Media3.
Development Tools	Git, Maven wrapper, Gradle wrapper, Node.js/npm, Android Studio or compatible IDE.

2.4. Assumptions and Dependencies

- Users need internet access for authentication, API access, and AI requests.
- The backend depends on a configured PostgreSQL database.
- Email verification and password recovery depend on a configured email sender.
- Google sign-in depends on a valid Google web client ID.
- AI features depend on configured AI model, proxy URL, and API key values.

3. System Features and Functional Requirements

Describe each major feature of the system and its functional requirements.

3.1. Feature 1: Authentication and Account Access

Description: The system allows guest users to create an account, verify identity, log in, refresh sessions, log out, use Google sign-in, and recover forgotten passwords.

Functional Requirements:
FR-AUTH-01: The system shall allow a guest user to register using username, email, and password.

FR-AUTH-02: The system shall support email verification for registered users.

FR-AUTH-03: The system shall allow a registered user to log in using valid credentials.

FR-AUTH-04: The system shall issue access and refresh tokens after successful login.

FR-AUTH-05: The system shall allow token refresh using a valid refresh token.

FR-AUTH-06: The system shall allow a user to log out and end the active session.

FR-AUTH-07: The system shall support forgot password, verification code, and reset password steps.

FR-AUTH-08: The system shall support Google sign-in using a valid Google identity token.

3.2. Feature 2: Profile and Settings Management

Description: The system allows authenticated users to view profile information and manage account settings.

Functional Requirements:

FR-PROF-01: The system shall allow an authenticated user to retrieve their profile.

FR-PROF-02: The system shall allow an authenticated user to update editable profile information.

FR-PROF-03: The system shall allow an authenticated user to change their password.

FR-PROF-04: The web application shall provide a Profile page that displays account information and account actions.

FR-PROF-05: The web application shall provide a Settings modal with Profile, Password, and AI Provider tabs.

FR-PROF-06: The system shall allow an authenticated user to configure AI provider settings from the Settings modal.

3.3. Feature 3: Notebook Management

Description: The system allows authenticated users to create, edit, review, organize, export, version, restore, and delete notebooks.

Functional Requirements:

FR-NOTE-01: The system shall allow an authenticated user to create a notebook.

FR-NOTE-02: The system shall allow an authenticated user to retrieve their notebook list.

FR-NOTE-03: The system shall allow an authenticated user to retrieve recently edited notebooks.

FR-NOTE-04: The system shall allow an authenticated user to retrieve recently reviewed notebooks.

FR-NOTE-05: The system shall allow an authenticated user to retrieve a notebook by UUID.

FR-NOTE-06: The system shall allow an authenticated user to update notebook metadata.

FR-NOTE-07: The system shall allow an authenticated user to save notebook content.

FR-NOTE-08: The system shall allow an authenticated user to update notebook review state.

FR-NOTE-09: The system shall create version snapshots when notebook content is saved.

FR-NOTE-10: The system shall allow an authenticated user to view notebook version history.

FR-NOTE-11: The system shall allow an authenticated user to preview a previous notebook version before restoration.

FR-NOTE-12: The system shall allow an authenticated user to delete a notebook they own.

FR-NOTE-13: The system shall allow an authenticated user to restore a previous notebook version.

FR-NOTE-14: The system shall allow an authenticated user to manually create a notebook version snapshot.

FR-NOTE-15: The system shall detect stale notebook content updates through version information and return a conflict response with the latest notebook data.

FR-NOTE-16: The web editor shall allow notebook content export to PDF/print, Word .docx, and text .txt formats.

FR-NOTE-17: The web editor shall provide formatting, table, math, page break, outline navigation, and review-mode controls where supported by the implemented editor.

3.4. Feature 4: Category Management

Description: The system allows authenticated users to organize notebooks using categories.

Functional Requirements:

FR-CAT-01: The system shall allow an authenticated user to retrieve categories.

FR-CAT-02: The system shall allow an authenticated user to create a category.

FR-CAT-03: The system shall allow an authenticated user to retrieve a category by ID.

FR-CAT-04: The system shall allow an authenticated user to delete a category.

FR-CAT-05: The system shall update affected notebooks when a category is deleted.

FR-CAT-06: The system shall allow notebook records to reference a user-owned category.

3.5. Feature 5: AI Configuration and Conversations

Description: The system allows authenticated users to configure AI access and use AI assistance in notebook workflows.

Functional Requirements:

FR-AI-01: The system shall allow an authenticated user to view the selected AI configuration.

FR-AI-02: The system shall allow an authenticated user to list saved AI configurations.

FR-AI-03: The system shall allow an authenticated user to create or update an AI configuration.

FR-AI-04: The system shall allow an authenticated user to select an AI configuration.

FR-AI-05: The system shall allow an authenticated user to delete an AI configuration.

FR-AI-06: The system shall allow an authenticated user to submit an AI query.

FR-AI-07: The system shall allow an authenticated user to list, create, update, and delete AI conversations for a notebook.

FR-AI-08: The system shall support AI-assisted notebook editing, review, quiz creation, and flashcard creation when configured.

FR-AI-09: The system shall support selected text and multiple AI selection targets in AI requests where provided by the editor.

FR-AI-10: The system shall keep AI provider keys server-side and prevent them from being exposed in client-visible source code.

3.6. Feature 6: Quiz Management and Study

Description: The system allows authenticated users to create quizzes, study them, and record quiz attempts.

Functional Requirements:

FR-QUIZ-01: The system shall allow an authenticated user to create a quiz.

FR-QUIZ-02: The system shall allow an authenticated user to list their quizzes.

FR-QUIZ-03: The system shall allow an authenticated user to retrieve a quiz by UUID.

FR-QUIZ-04: The system shall allow an authenticated user to update a quiz.

FR-QUIZ-05: The system shall allow an authenticated user to delete a quiz.

FR-QUIZ-06: The system shall allow an authenticated user to record quiz attempts and scores.

FR-QUIZ-07: The system shall associate quizzes with notebooks when a notebook UUID is provided.

FR-QUIZ-08: The web and mobile clients shall allow users to study quizzes and submit attempts.

3.7. Feature 7: Flashcard Management and Study

Description: The system allows authenticated users to create flashcard decks, study them, and record flashcard attempts.

Functional Requirements:

FR-FLASH-01: The system shall allow an authenticated user to create a flashcard deck.

FR-FLASH-02: The system shall allow an authenticated user to list their flashcard decks.

FR-FLASH-03: The system shall allow an authenticated user to retrieve a flashcard deck by UUID.

FR-FLASH-04: The system shall allow an authenticated user to update a flashcard deck.

FR-FLASH-05: The system shall allow an authenticated user to delete a flashcard deck.

FR-FLASH-06: The system shall allow an authenticated user to record flashcard attempts and mastery.

FR-FLASH-07: The system shall associate flashcard decks with notebooks when a notebook UUID is provided.

FR-FLASH-08: The web and mobile clients shall allow users to study flashcard decks and submit mastery attempts.

3.8. Feature 8: Playlist Management

Description: The system allows authenticated users to organize notebooks into ordered study playlists.

Functional Requirements:

FR-PLAYLIST-01: The system shall allow an authenticated user to create a playlist.

FR-PLAYLIST-02: The system shall allow an authenticated user to list playlists.

FR-PLAYLIST-03: The system shall allow an authenticated user to retrieve a playlist by UUID.

FR-PLAYLIST-04: The system shall allow an authenticated user to update playlist metadata.

FR-PLAYLIST-05: The system shall allow an authenticated user to delete a playlist.

FR-PLAYLIST-06: The system shall allow an authenticated user to add notebooks to a playlist.

FR-PLAYLIST-07: The system shall allow an authenticated user to remove notebooks from a playlist.

FR-PLAYLIST-08: The system shall allow an authenticated user to reorder playlist notebooks.

FR-PLAYLIST-09: The system shall allow an authenticated user to update the playlist current index.

3.9. Feature 9: Playback Queue and Audio Study

Description: The system allows authenticated users to build and control a notebook playback queue.

Functional Requirements:

FR-QUEUE-01: The system shall allow an authenticated user to retrieve the current playback queue.

FR-QUEUE-02: The system shall allow an authenticated user to add notebooks to the current playback queue.

FR-QUEUE-03: The system shall allow an authenticated user to select a playlist as the current queue source.

FR-QUEUE-04: The system shall allow an authenticated user to remove notebooks from the current playback queue.

FR-QUEUE-05: The system shall allow an authenticated user to clear the current playback queue.

FR-QUEUE-06: The system shall allow an authenticated user to update the queue current index.

FR-QUEUE-07: The system shall allow an authenticated user to reorder the queue.

FR-QUEUE-08: The mobile application shall provide playback controls for play, pause, skip, loop, shuffle, speech rate, playback position resume, and notebook review-state updates where supported.

FR-QUEUE-09: The web application shall provide a player bar and queue panel for current queue visibility and playback control.

FR-QUEUE-10: The system shall keep playlist and playback queue endpoints consistent through supported alias paths.

4. Non-Functional Requirements

NFR-01: The system shall require authenticated access for protected API endpoints.

NFR-02: The system shall enforce ownership so users can only access records they own.

NFR-03: The system shall store passwords using one-way encryption or hashing.

NFR-04: The system shall keep refresh token sessions with expiry information.

NFR-05: The system shall keep AI keys and email credentials outside client-visible source code.

NFR-06: The web application shall run in modern browsers supported by the React/Vite toolchain.

NFR-07: The mobile application shall support Android API level 24 and higher.

NFR-08: The system shall provide clear navigation for authentication, dashboard, library, notebook editor, quizzes, flashcards, playlists, playback, profile, and settings workflows.

NFR-10: The system shall provide reliable error messages for invalid input, unauthorized access, missing records, route load failures, and failed external service requests.

NFR-11: The system shall support automated testing for major backend, web, and mobile features.

NFR-12: The system shall keep external service credentials and environment-specific values configurable outside the client-visible application code.

NFR-13: The system shall use version information to protect notebook saves from stale-client overwrites.

NFR-14: The system shall use client mutation IDs where implemented to avoid duplicate effects for repeated notebook, quiz, or flashcard attempt requests.

NFR-15: The Android application shall keep feature boundaries between app orchestration, feature slices, platform networking/session code, and shared UI/study components.

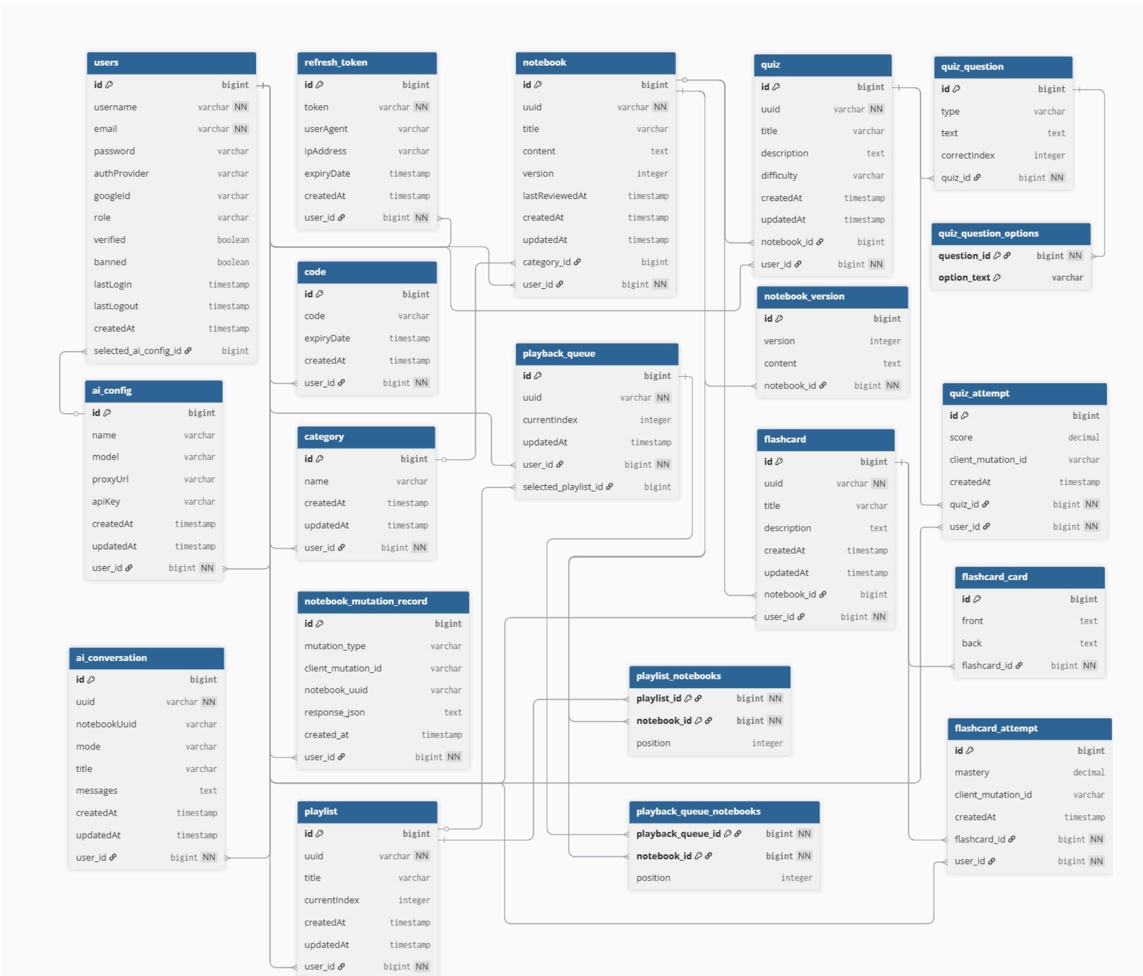
NFR-16: The web application shall provide route-level fallback behavior for lazy-loaded pages and route errors.

NFR-17: Mobile playback shall split notebook text into manageable text-to-speech chunks and preserve resume information where supported.

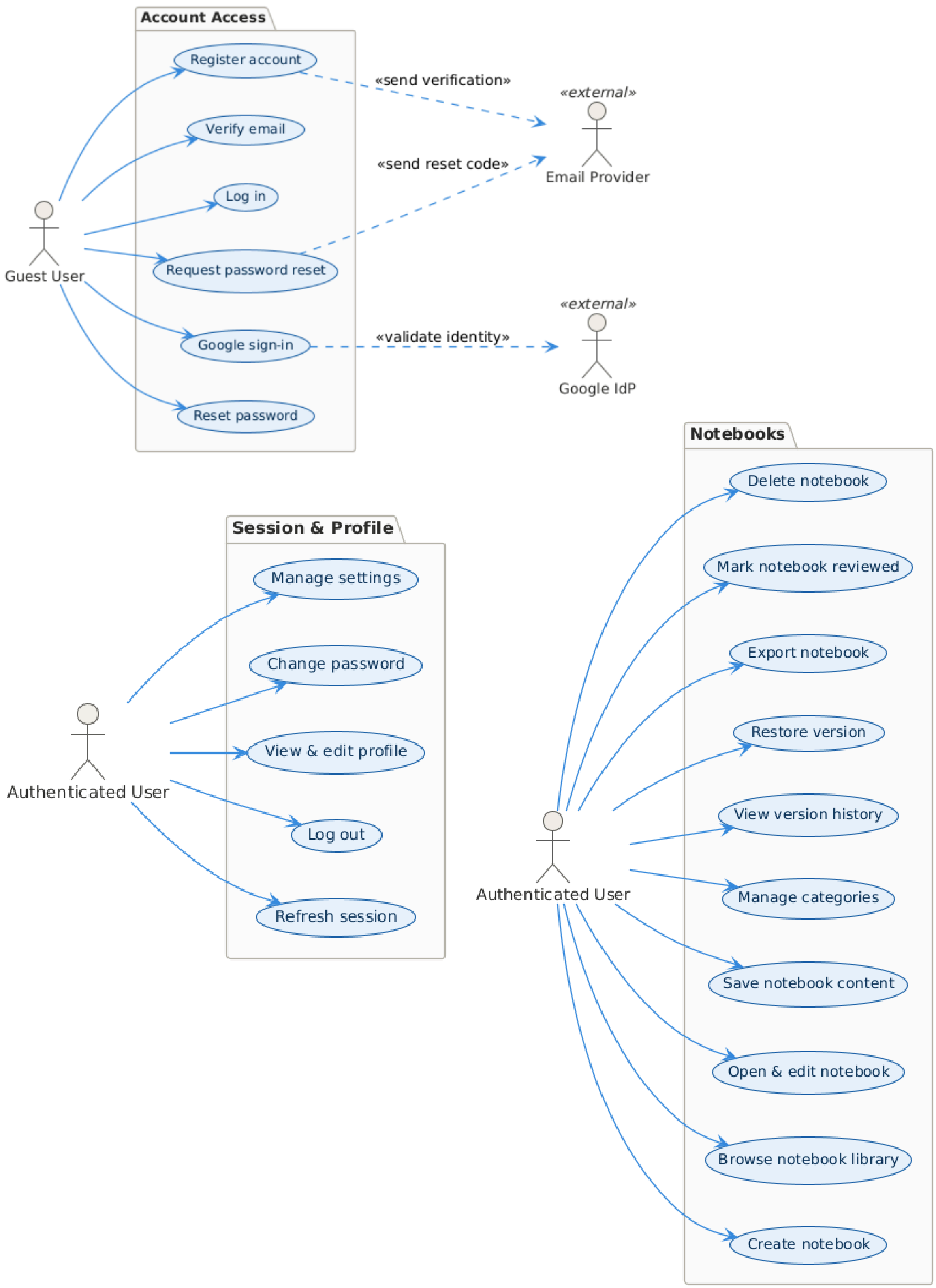
5. System Models (Diagrams)

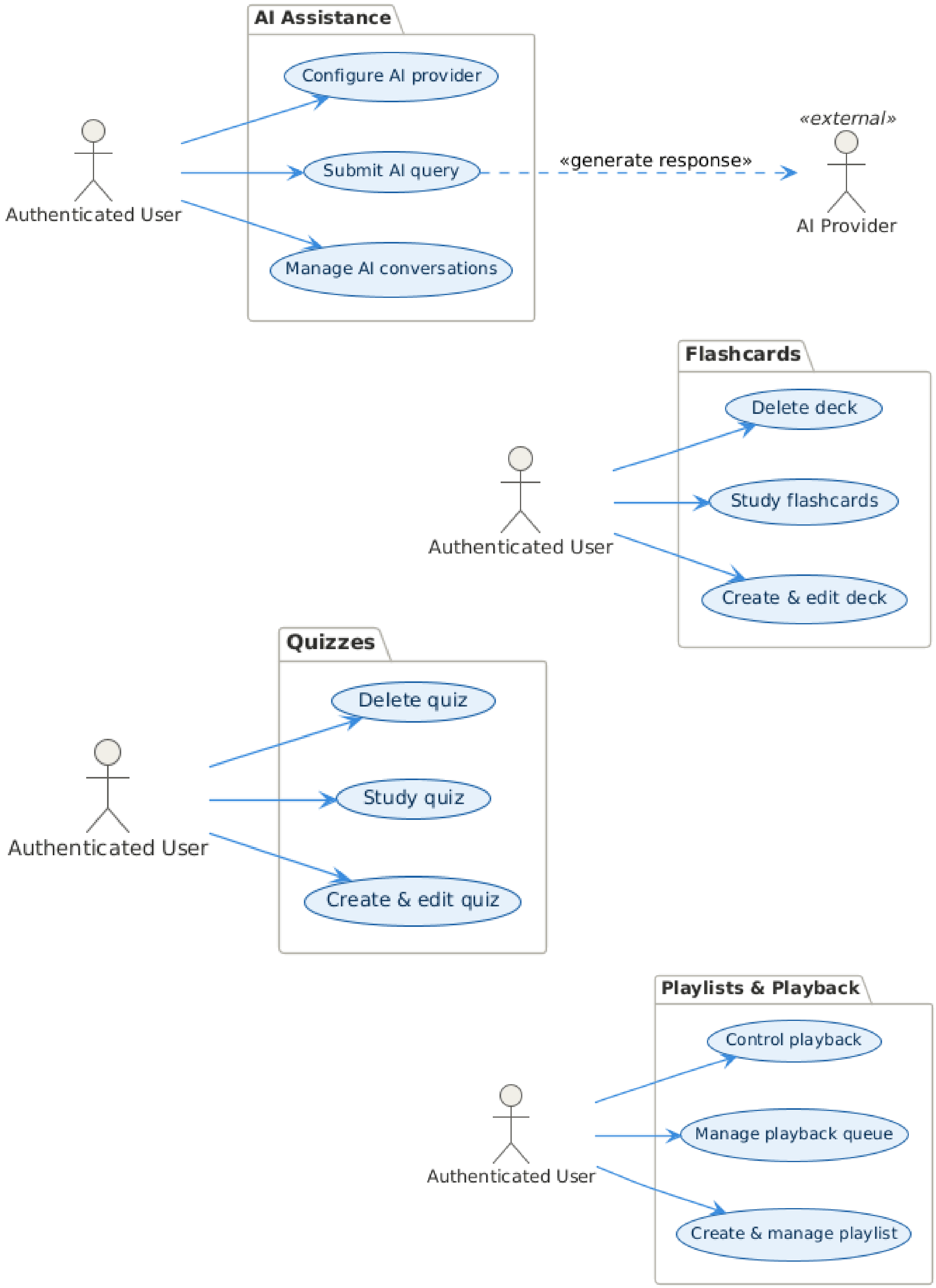
Insert the necessary diagrams for the system:

5.1. ERD

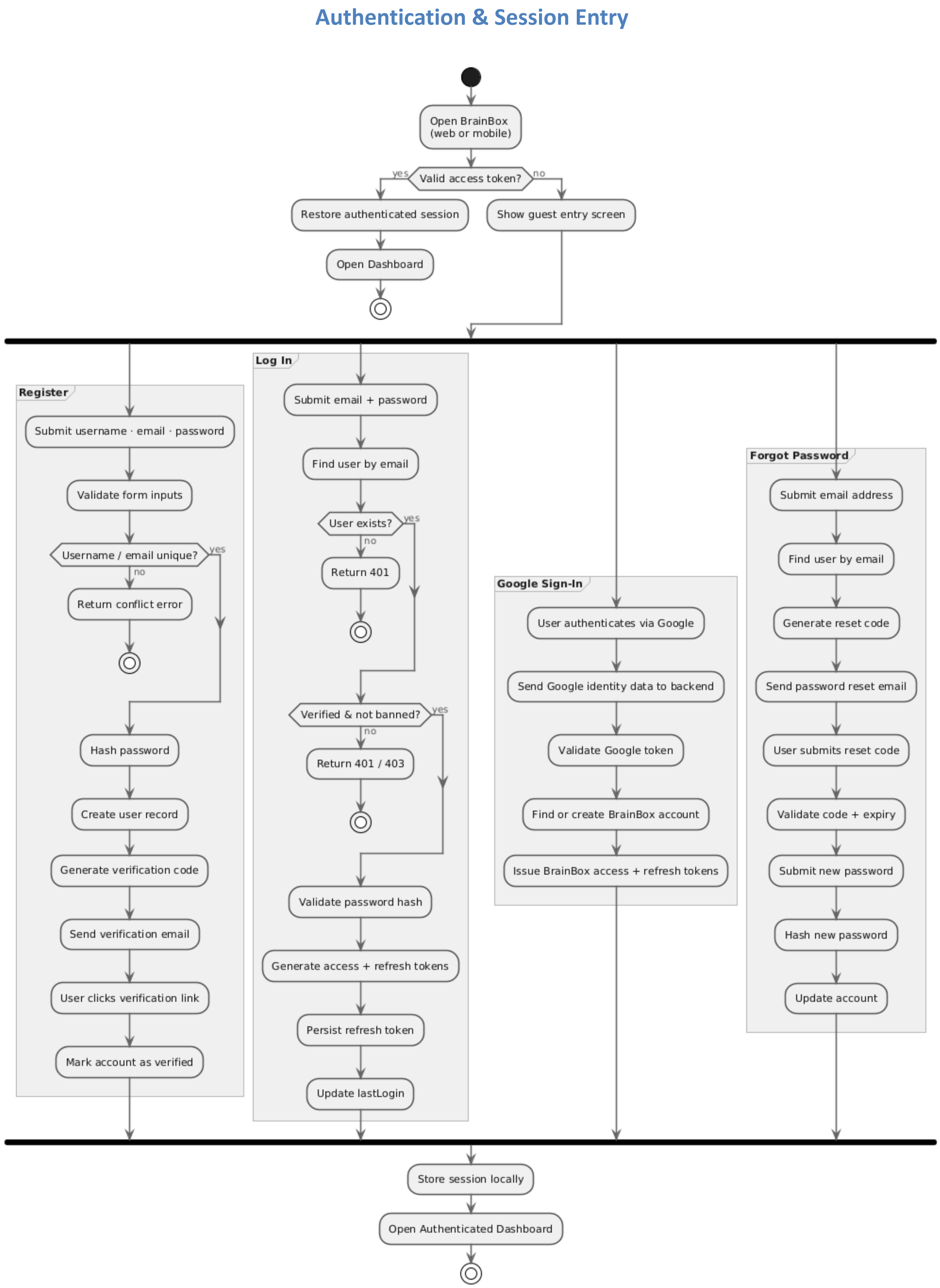


5.2. Use Case Diagram

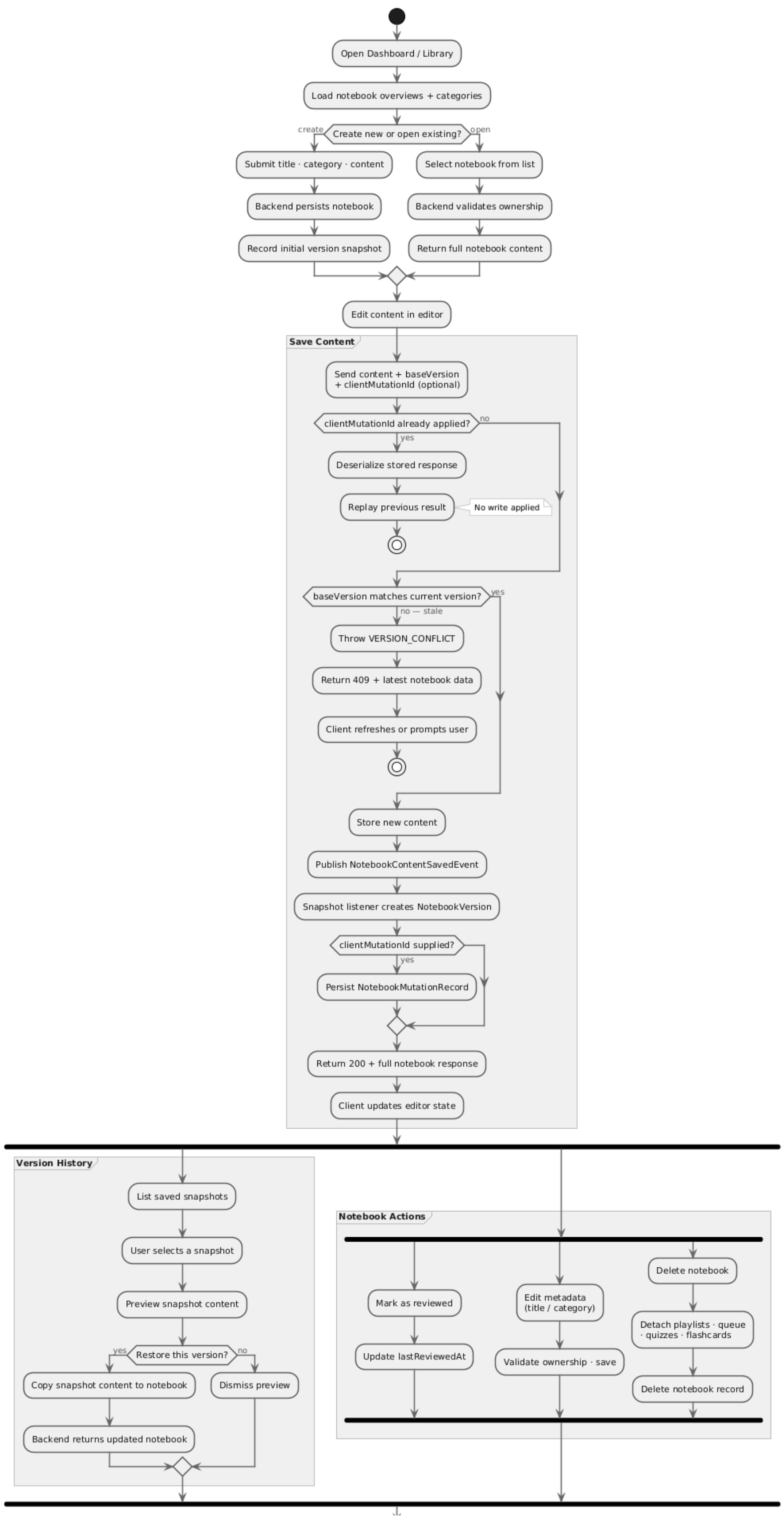



AI Assistance
Configure AI provider
Submit AI query
Manage AI conversations
Authenticated User
«generate response»
«external»
AI Provider
Flashcards
Delete deck
Study flashcards
Create & edit deck
Authenticated User
Quizzes
Delete quiz
Study quiz
Create & edit quiz
Authenticated User
Playlists & Playback
Control playback
Manage playback queue
Create & manage playlist
Authenticated User

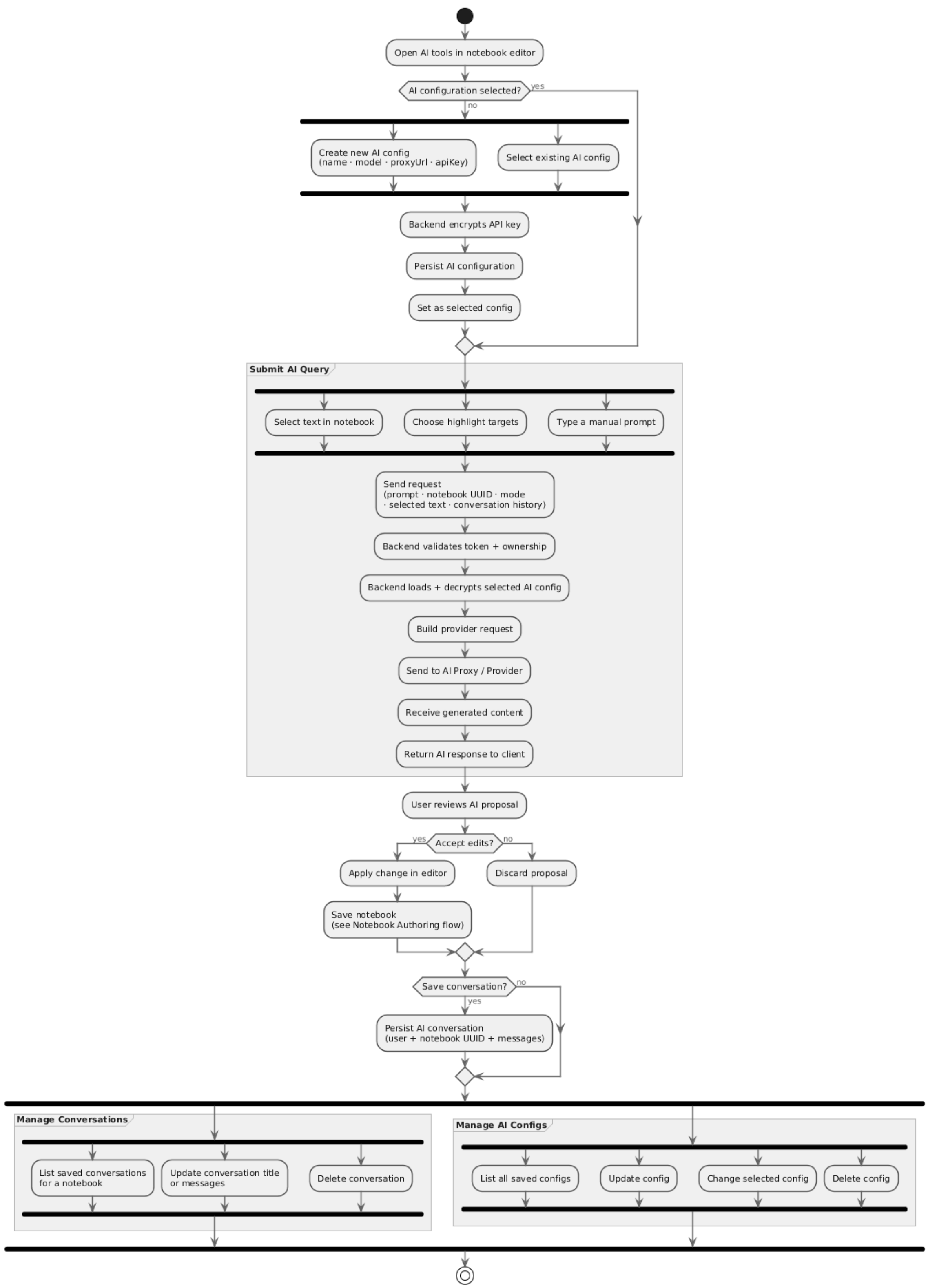
5.3. Activity Diagram



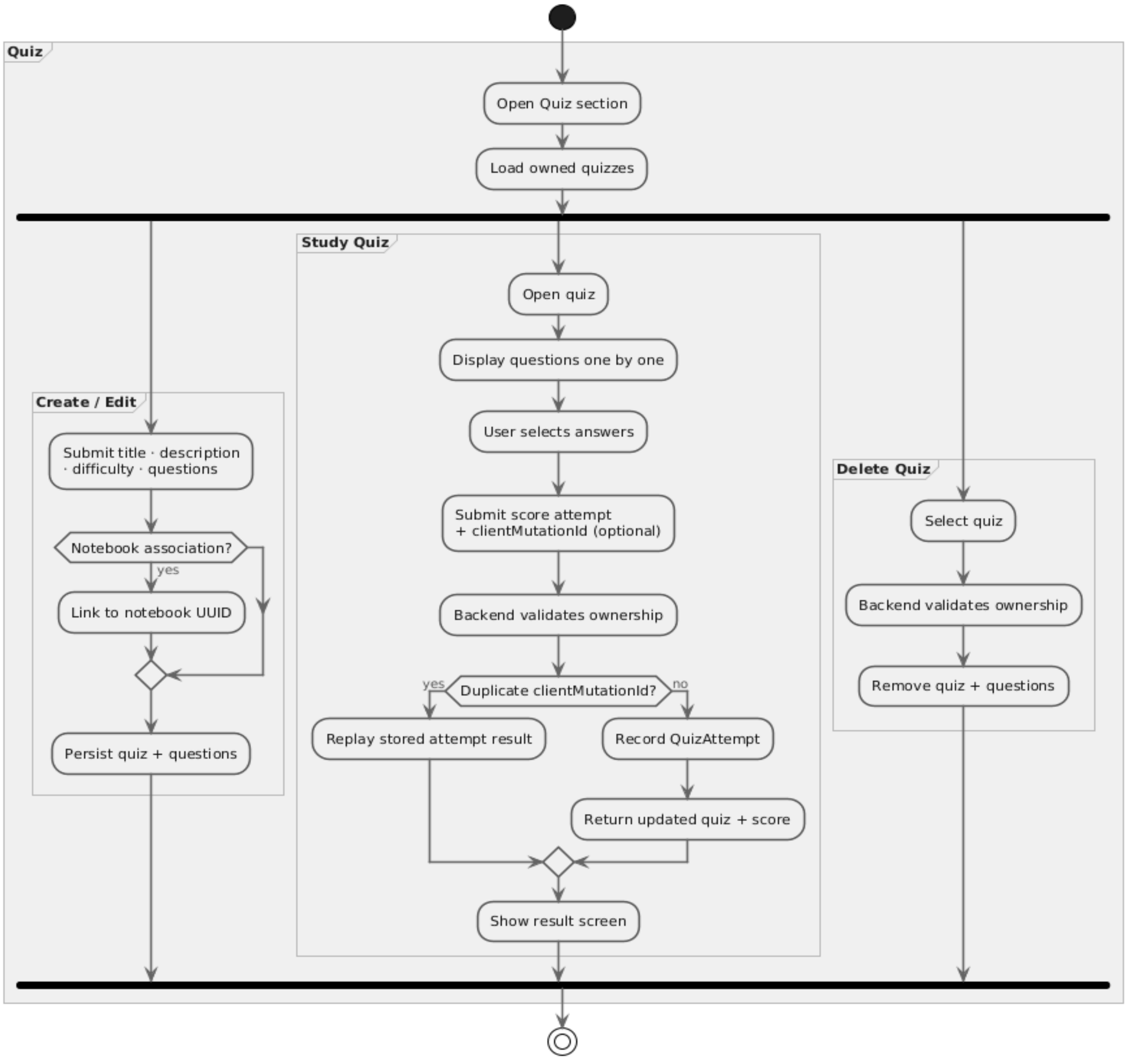
Notebook Authoring & Recovery



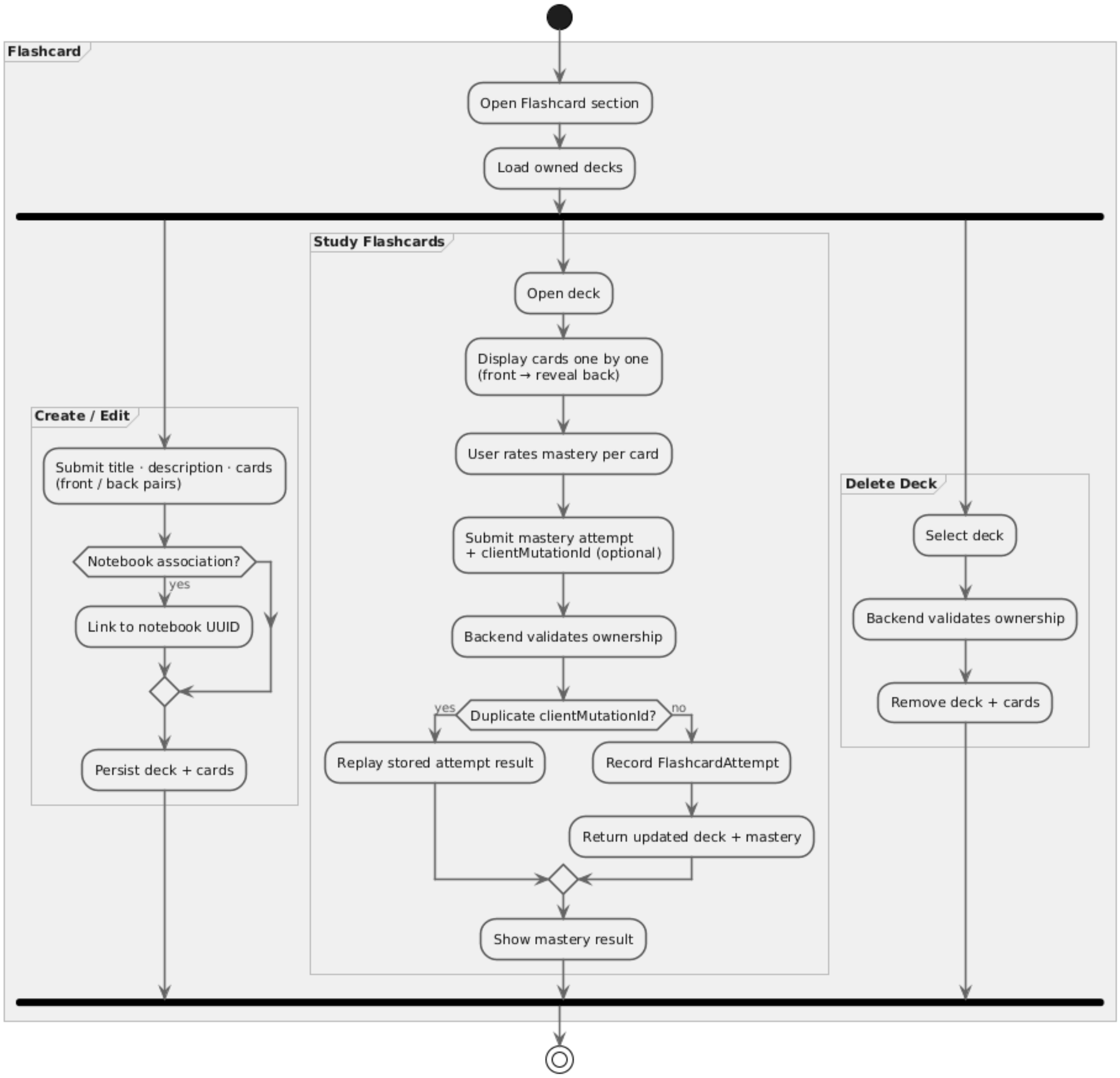
AI Assistance



Quiz

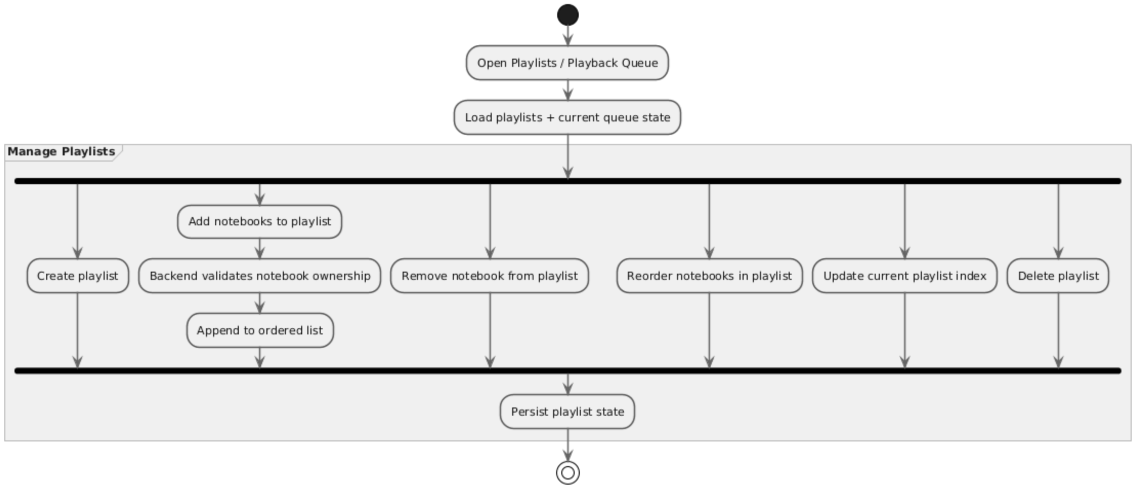

Quiz
Open Quiz section
Load owned quizzes
Create / Edit
Submit title · description · difficulty · questions
Notebook association?
yes
Link to notebook UUID
Persist quiz + questions
Study Quiz
Open quiz
Display questions one by one
User selects answers
Submit score attempt + clientMutationId (optional)
Backend validates ownership
Duplicate clientMutationId?
yes
no
Replay stored attempt result
Record QuizAttempt
Return updated quiz + score
Show result screen
Delete Quiz
Select quiz
Backend validates ownership
Remove quiz + questions

Flashcards

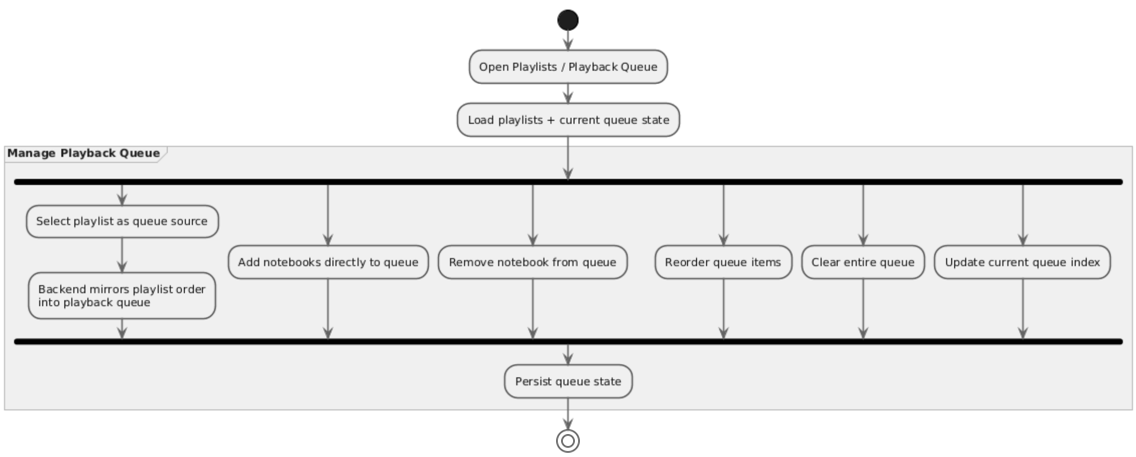


Flashcard
Open Flashcard section
Load owned decks
Create / Edit
Submit title · description · cards
(front / back pairs)
Notebook association?
yes
Link to notebook UUID
Persist deck + cards
Study Flashcards
Open deck
Display cards one by one
(front → reveal back)
User rates mastery per card
Submit mastery attempt
+ clientMutationId (optional)
Backend validates ownership
Duplicate clientMutationId?
yes
no
Replay stored attempt result
Record FlashcardAttempt
Return updated deck + mastery
Show mastery result
Delete Deck
Select deck
Backend validates ownership
Remove deck + cards

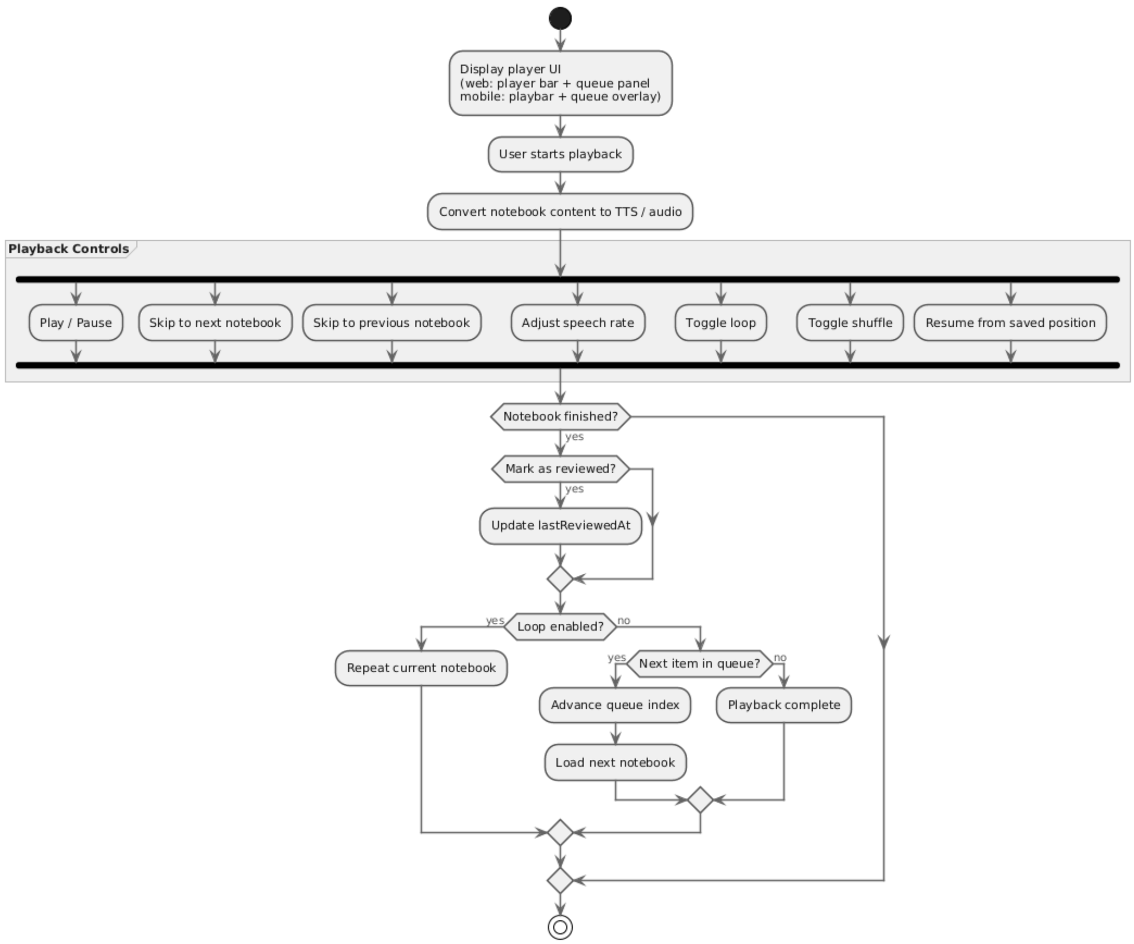
Manage Playlists



Manage Playlist Queue

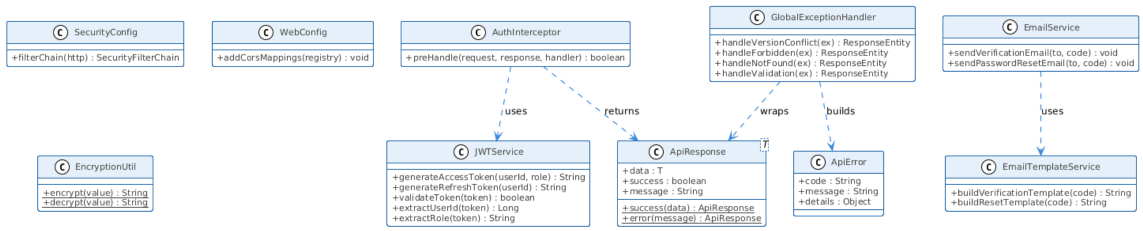


Playback Controls

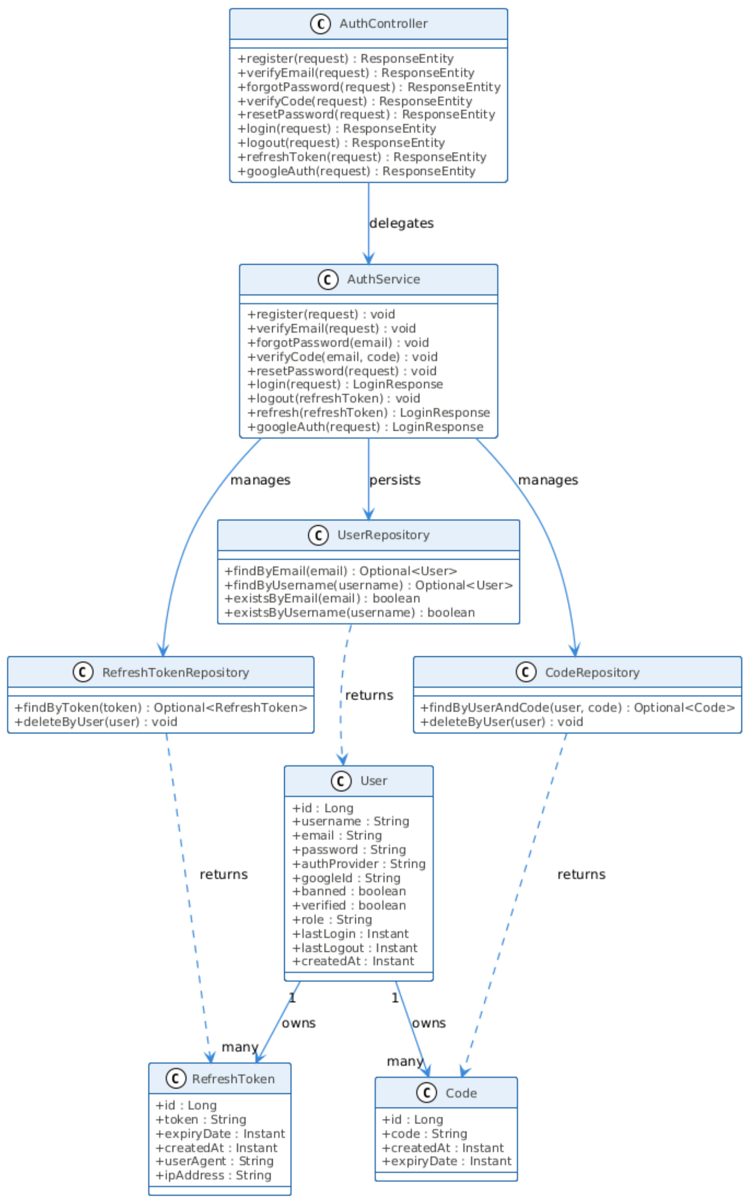


5.4. Class Diagram

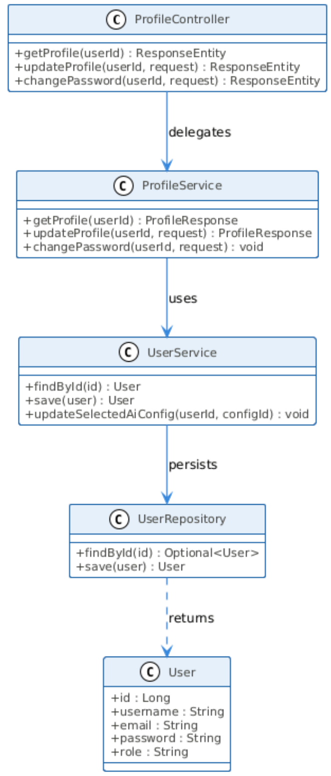
Platform & Shared Layer



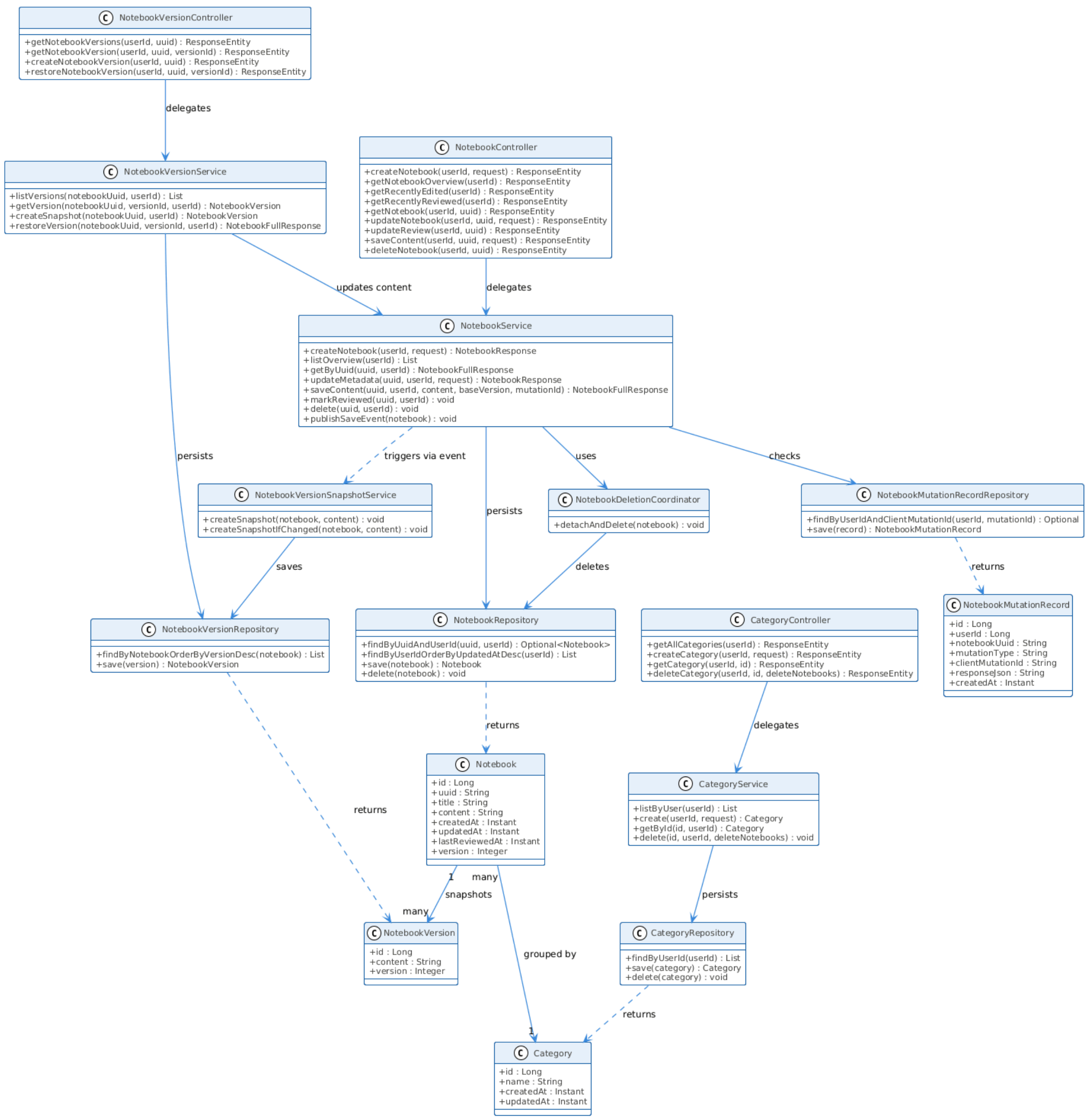
Authentication



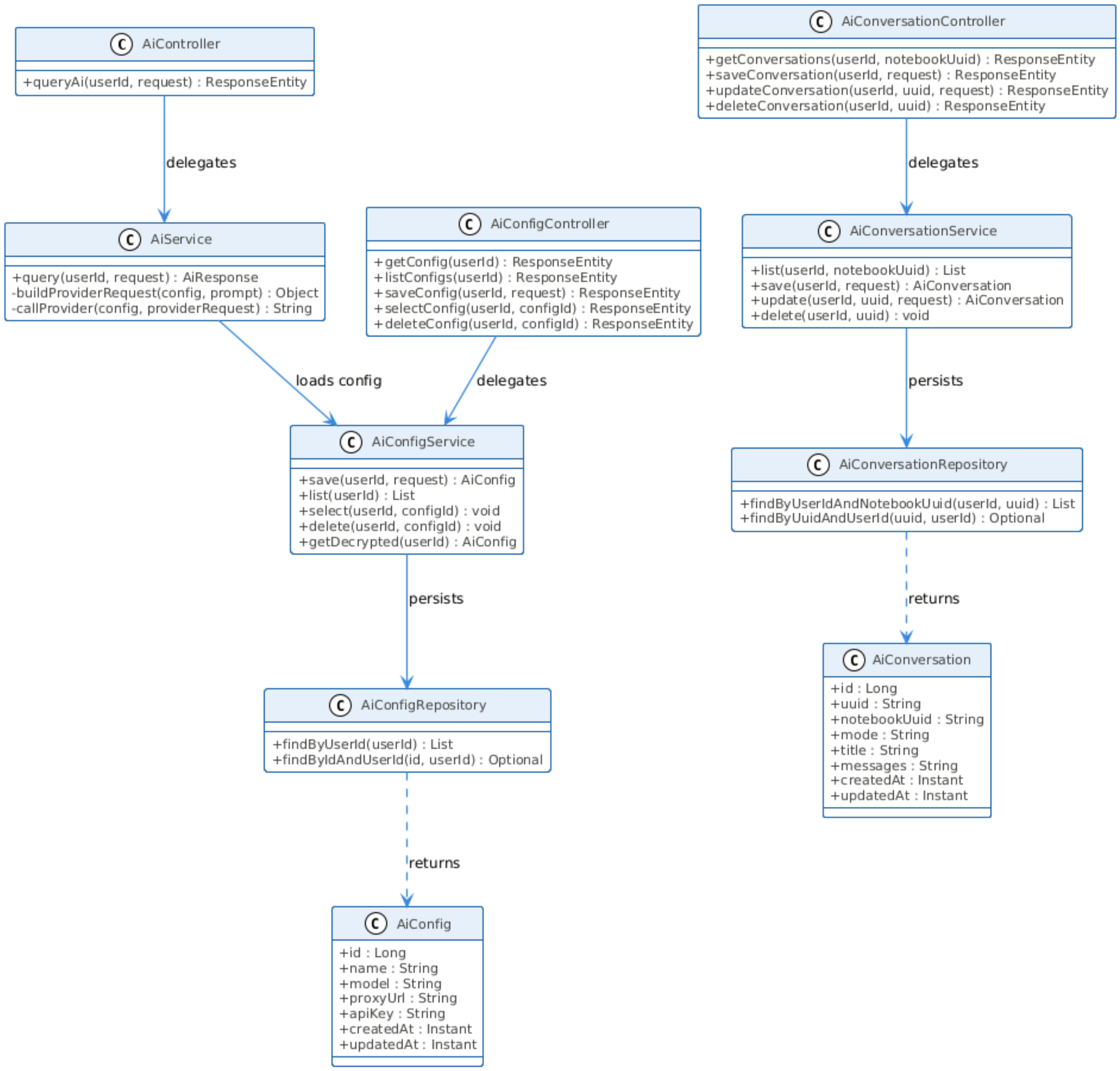
Profile


ProfileController
+getProfile(userId) : ResponseEntity
+updateProfile(userId, request) : ResponseEntity
+changePassword(userId, request) : ResponseEntity
delegates
ProfileService
+getProfile(userId) : ProfileResponse
+updateProfile(userId, request) : ProfileResponse
+changePassword(userId, request) : void
uses
UserService
+findById(id) : User
+save(user) : User
+updateSelectedAiConfig(userId, configId) : void
persists
UserRepository
+findById(id) : Optional<User>
+save(user) : User
returns
User
+id : Long
+username : String
+email : String
+password : String
+role : String

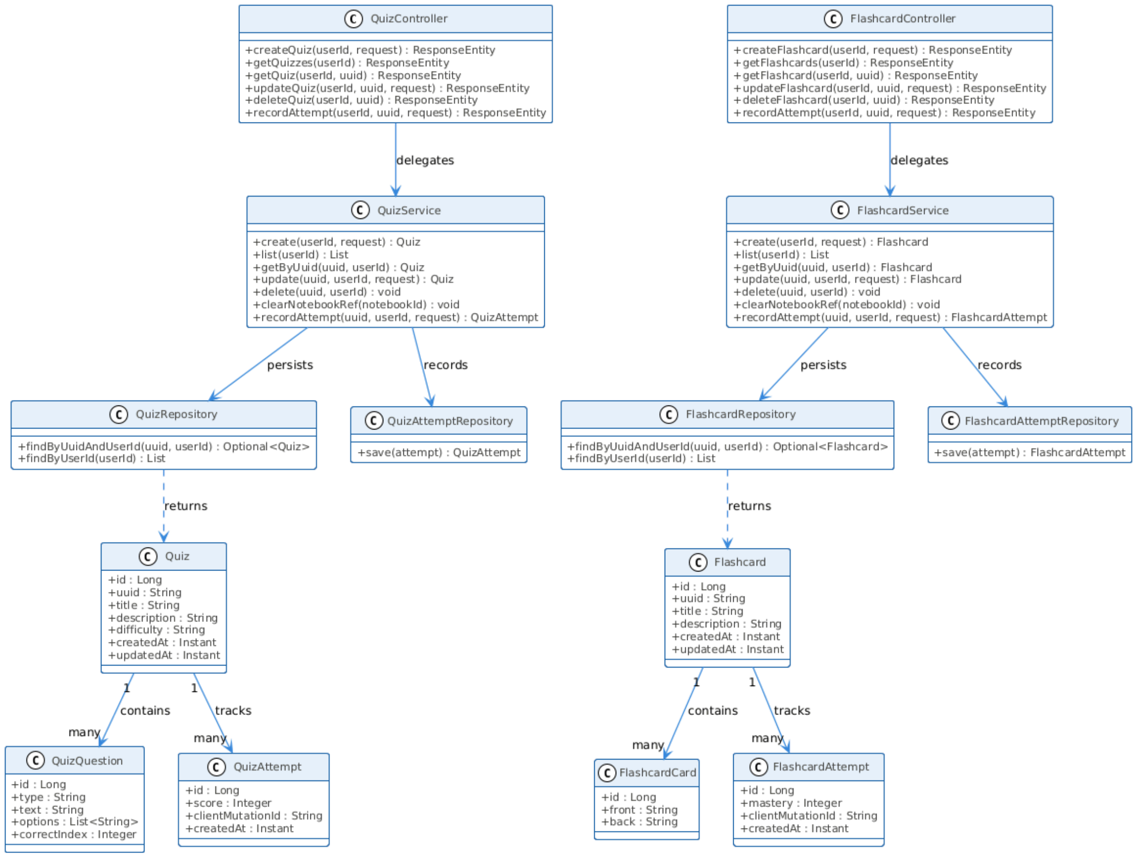
Notebooks


NotebookVersionController
+getNotebookVersions(userId, uuid) : ResponseEntity
+getNotebookVersion(userId, uuid, versionId) : ResponseEntity
+createNotebookVersion(userId, uuid) : ResponseEntity
+restoreNotebookVersion(userId, uuid, versionId) : ResponseEntity
delegates
NotebookVersionService
+listVersions(notebookUuid, userId) : List
+getVersion(notebookUuid, versionId, userId) : NotebookVersion
+createSnapshot(notebookUuid, userId) : NotebookVersion
+restoreVersion(notebookUuid, versionId, userId) : NotebookFullResponse
NotebookController
+createNotebook(userId, request) : ResponseEntity
+getNotebookOverview(userId) : ResponseEntity
+getRecentlyEdited(userId) : ResponseEntity
+getRecentlyReviewed(userId) : ResponseEntity
+getNotebook(userId, uuid) : ResponseEntity
+updateNotebook(userId, uuid, request) : ResponseEntity
+updateReview(userId, uuid) : ResponseEntity
+saveContent(userId, uuid, request) : ResponseEntity
+deleteNotebook(userId, uuid) : ResponseEntity
updates content
delegates
NotebookService
+createNotebook(userId, request) : NotebookResponse
+listOverview(userId) : List
+getByUuid(uuid, userId) : NotebookFullResponse
+updateMetadata(uuid, userId, request) : NotebookResponse
+saveContent(uuid, userId, content, baseVersion, mutationId) : NotebookFullResponse
+markReviewed(uuid, userId) : void
+delete(uuid, userId) : void
+publishSaveEvent(notebook) : void
persists
triggers via event
uses
checks
NotebookVersionSnapshotService
+createSnapshot(notebook, content) : void
+createSnapshotIfChanged(notebook, content) : void
persists
NotebookDeletionCoordinator
+detachAndDelete(notebook) : void
NotebookMutationRecordRepository
+findByUserIdAndClientMutationId(userId, mutationId) : Optional
+save(record) : NotebookMutationRecord
saves
deletes
returns
NotebookVersionRepository
+findByNotebookOrderByVersionDesc(notebook) : List
+save(version) : NotebookVersion
NotebookRepository
+findByUuidAndUserId(uuid, userId) : Optional<Notebook>
+findByUserIdOrderByUpdatedAtDesc(userId) : List
+save(notebook) : Notebook
+delete(notebook) : void
CategoryController
+getAllCategories(userId) : ResponseEntity
+createCategory(userId, request) : ResponseEntity
+getCategory(userId, id) : ResponseEntity
+deleteCategory(userId, id, deleteNotebooks) : ResponseEntity
NotebookMutationRecord
+id : Long
+userId : Long
+notebookUuid : String
+mutationType : String
+clientMutationId : String
+responseJson : String
+createdAt : Instant
returns
delegates
Notebook
+id : Long
+uuid : String
+title : String
+content : String
+createdAt : Instant
+updatedAt : Instant
+lastReviewedAt : Instant
+version : Integer
returns
CategoryService
+listByUser(userId) : List
+create(userId, request) : Category
+getById(id, userId) : Category
+delete(id, userId, deleteNotebooks) : void
1
many
snapshots
many
persists
NotebookVersion
+id : Long
+content : String
+version : Integer
CategoryRepository
+findByUserId(userId) : List
+save(category) : Category
+delete(category) : void
grouped by
returns
1
Category
+id : Long
+name : String
+createdAt : Instant
+updatedAt : Instant

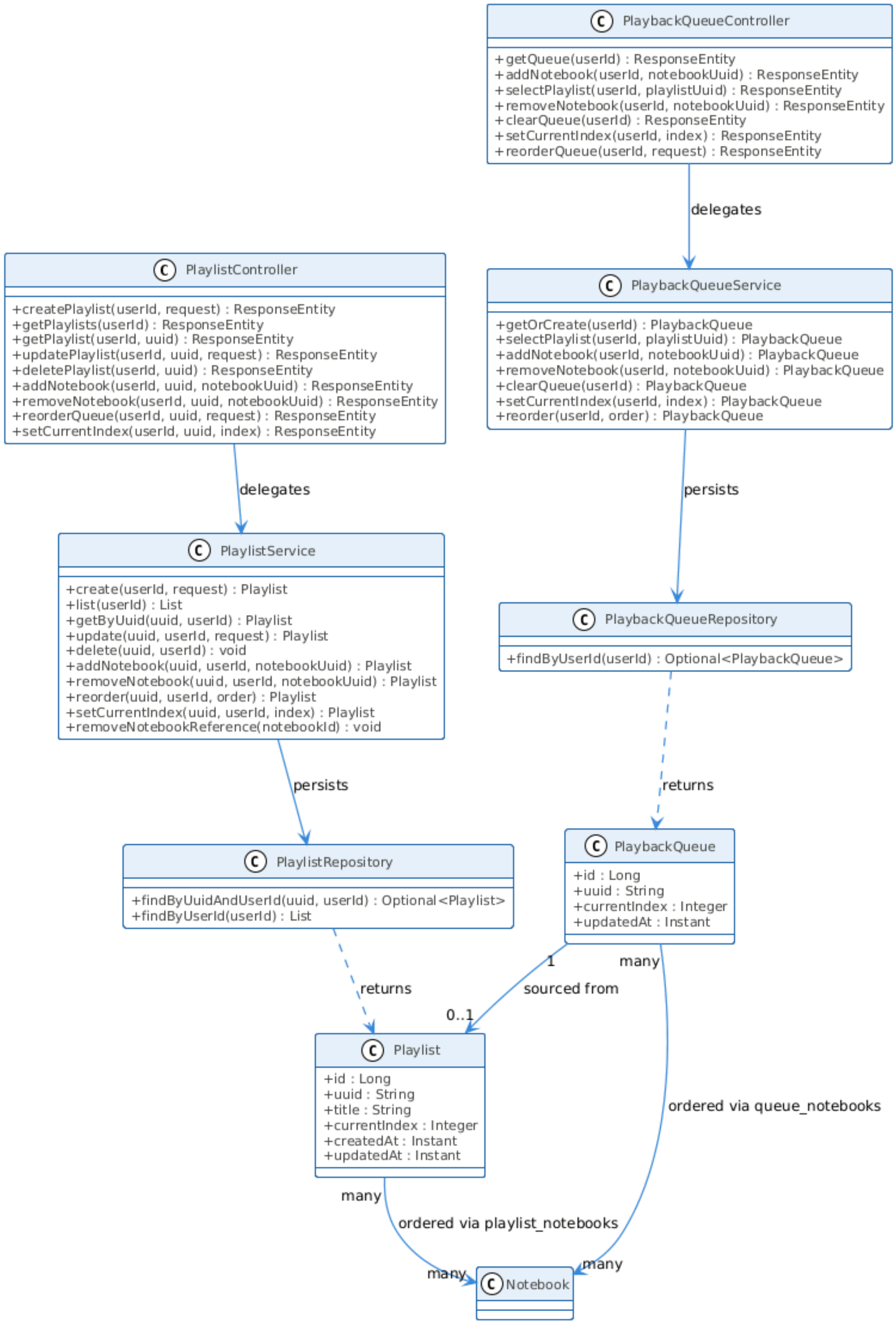
AI


AiController
+queryAi(userId, request) : ResponseEntity
delegates
AiService
+query(userId, request) : AiResponse
-buildProviderRequest(config, prompt) : Object
-callProvider(config, providerRequest) : String
loads config
AiConfigController
+getConfig(userId) : ResponseEntity
+listConfigs(userId) : ResponseEntity
+saveConfig(userId, request) : ResponseEntity
+selectConfig(userId, configId) : ResponseEntity
+deleteConfig(userId, configId) : ResponseEntity
delegates
AiConfigService
+save(userId, request) : AiConfig
+list(userId) : List
+select(userId, configId) : void
+delete(userId, configId) : void
+getDecrypted(userId) : AiConfig
persists
AiConfigRepository
+findByUserId(userId) : List
+findByIdAndUserId(id, userId) : Optional
returns
AiConfig
+id : Long
+name : String
+model : String
+proxyUrl : String
+apiKey : String
+createdAt : Instant
+updatedAt : Instant
AiConversationController
+getConversations(userId, notebookUuid) : ResponseEntity
+saveConversation(userId, request) : ResponseEntity
+updateConversation(userId, uuid, request) : ResponseEntity
+deleteConversation(userId, uuid) : ResponseEntity
delegates
AiConversationService
+list(userId, notebookUuid) : List
+save(userId, request) : AiConversation
+update(userId, uuid, request) : AiConversation
+delete(userId, uuid) : void
persists
AiConversationRepository
+findByUserIdAndNotebookUuid(userId, uuid) : List
+findByUuidAndUserId(uuid, userId) : Optional
returns
AiConversation
+id : Long
+uuid : String
+notebookUuid : String
+mode : String
+title : String
+messages : String
+createdAt : Instant
+updatedAt : Instant

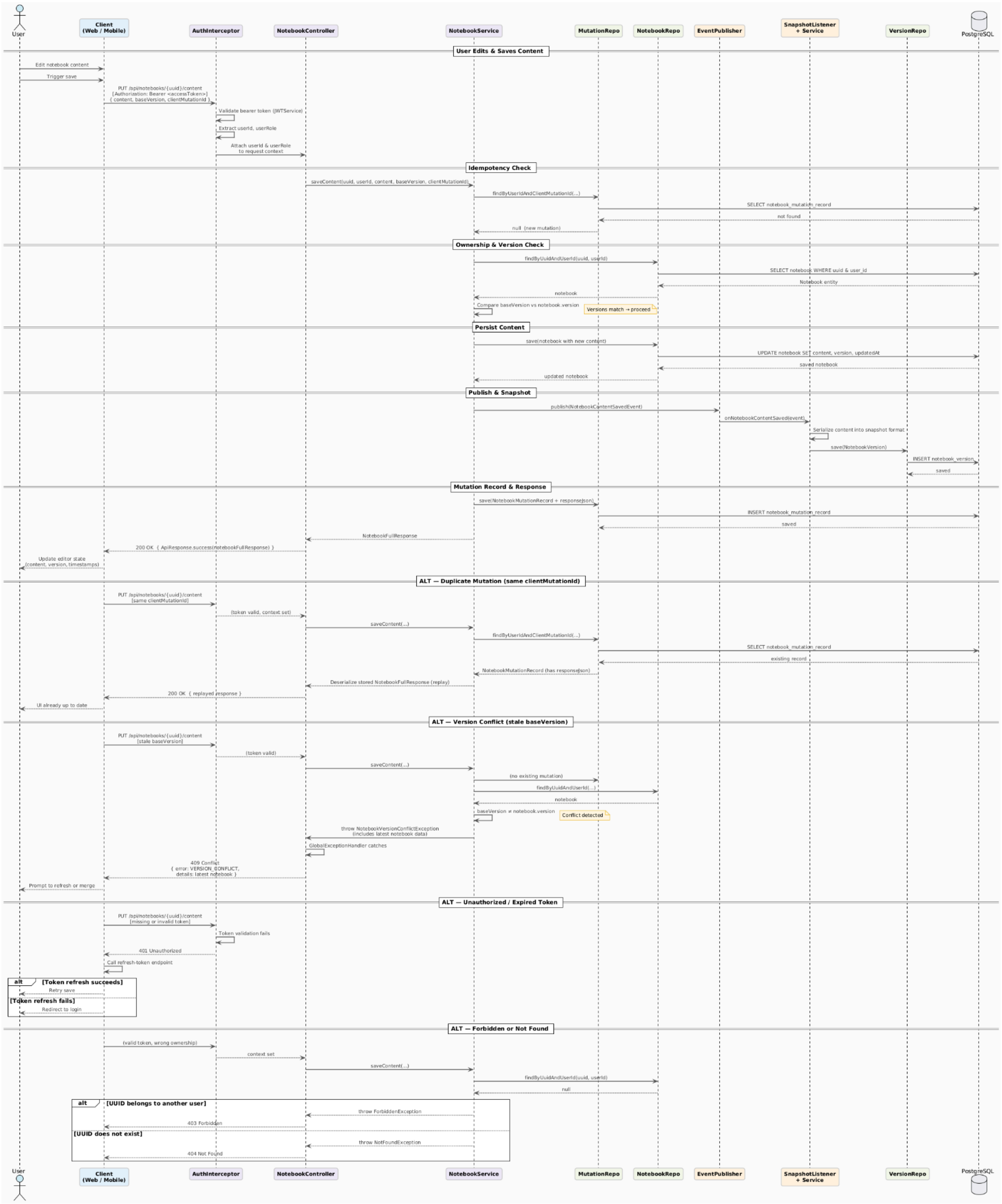
Quizzes & Flashcards



Playlists & Playback Queue



5.5. Sequence Diagram



6. Appendices

Include any additional information, references, or support materials.